

PROMETEO



Aplicación Móvil de análisis biomecánico de sentadillas
mediante visión artificial

Grado superior en Desarrollo de Aplicaciones

Multiplataforma

Alumno: Manuel Lorenzo Zoreda

Tutor del TFG: Luz María Álvarez Moreno

ÍNDICE DE CONTENIDOS

RESUMEN.....	5
ABSTRACT.....	5
PALABRAS CLAVE.....	5
JUSTIFICACIÓN DEL PROYECTO.....	7
Motivación.....	7
Estado de la cuestión.....	7
Público objetivo y necesidades cubiertas.....	8
Justificación técnica.....	8
ANTECEDENTES.....	10
Análisis del estado del arte.....	10
Tabla comparativa de soluciones.....	11
INTRODUCCIÓN.....	12
Descripción general.....	12
Problemas que resuelve.....	13
Alcance del proyecto.....	13
OBJETIVOS.....	14
1. RFTP: Gestión del ciclo de vida de la sesión.....	14
2. RFTP: Subida de video y orquestación del análisis.....	15
3. RFTP: Análisis biomecánico con visión artificial.....	16
4. RFTP: Interfaz de usuario Android (Jetpack Compose).....	18

PROMETEO

DESCRIPCIÓN.....	19
Arquitectura del sistema.....	19
Flujo de datos detallado.....	20
Descripción de los módulos.....	23
Casos de uso.....	26
DISEÑOS.....	32
TECNOLOGÍA.....	37
Descripción de tecnologías clave.....	38
METODOLOGÍA.....	40
Metodología empleada.....	40
Planificación inicial estimada.....	41
Planificación real ejecutada.....	42
Presupuesto del proyecto.....	43
DESPLIEGUE E INSTALACIÓN.....	44
Requisitos previos del sistema.....	44
Instalación del servicio Python.....	44
Instalación del backend Java.....	45
Configuración de la app Android.....	45
Verificación y planes de contingencia.....	46
TRABAJOS FUTUROS.....	47
CONCLUSIONES.....	49

PROMETEO

REFERENCIAS.....	21
------------------	----

PROMETEO

RESUMEN

Este proyecto consiste en el desarrollo de una aplicación en la que se analiza la biomecánica del movimiento de sentadilla mediante visión artificial a partir de un video.

En el video se establecerán unos puntos en las articulaciones de cadera, rodillas y tobillos para comprobar la profundidad de la sentadilla y si esta es correcta, es decir que el ángulo de la cadera corte a la paralela de las rodillas (estándar powerlifting), contar una repetición válida, en caso contrario la repetición será no válida. Además, se implementa una estimación de la velocidad de ejecución del movimiento, con esta métrica se puede estimar la fatiga del usuario y la carga máxima a levantar.

ABSTRACT

This project involves the development of an application that analyzes the biomechanics of the squat movement using computer vision based on video footage.

In the video, points will be established at the hip, knee, and ankle joints to verify the depth of the squat and whether it is correct—that is, whether the hip angle intersects the line parallel to the knees (powerlifting standard)—to count a valid repetition; otherwise, the repetition will be invalid.

Additionally, an estimate of the movement's execution speed is implemented; this metric can be used to estimate the user's fatigue and the maximum load to be lifted.

PALABRAS CLAVE

PROMETEO

Visión artificial, análisis biomecánico, sentadilla, powerlifting, MediaPipe, Android, Spring Boot, FastAPI, estimación 1RM, índice de fatiga, Kotlin, Jetpack Compose.

JUSTIFICACIÓN DEL PROYECTO

Motivación

El entrenamiento de fuerza y a su vez el powerlifting han experimentado un crecimiento exponencial en los últimos años. La sentadilla, siendo uno de los tres movimientos de esta disciplina además de un patrón de movimiento indispensable para cualquier entrenamiento, requiere de una técnica precisa para maximizar el rendimiento y minimizar el riesgo de lesiones.

Actualmente, el análisis técnico de este ejercicio depende casi exclusivamente de entrenadores cualificados o de la autoevaluación mediante grabaciones, algo subjetivo y poco preciso. Los deportistas no profesionales carecen de herramientas accesibles que les proporcionen feedback objetivo sobre su técnica, más allá de observaciones generales.

De esta necesidad nace este proyecto, ofreciendo una herramienta que utiliza visión artificial para proporcionar métricas cuantificables y feedback técnico basado en los estándares oficiales de powerlifting, aunque aplicable a cualquier tipo de entrenamiento.

Estado de la cuestión

Actualmente existen algunas soluciones en el mercado que abordan este problema, pero de forma parcial, siendo de pago o necesitando de hardware adicional como podría ser un encoder.

Tras un estudio, la ventaja diferencial de este proyecto frente al resto de soluciones es que ninguna de estas combina validación reglamentaria de profundidad, análisis de velocidad, verticalidad de la barra y conteo automático en una única solución.

PROMETEO

Público objetivo

- Practicantes de powerlifting de cualquier nivel
- Personas que entrenen fuerza y quieran mejorar su rendimiento en este movimientos
- Entrenadores personales que buscan nuevas herramientas de evaluación
- Preparadores físicos que trabajen con múltiples clientes de distintos perfiles

Necesidades cubiertas

- Validación objetiva de si las repeticiones del movimiento serían válidas en competición (ángulo de rodilla $\leq 90^\circ$)
- Feedback cuantificable para corrección técnica progresiva
- Estimación de carga máxima para individualización de entrenamientos

Justificación técnica

La elección de visión artificial con MediaPipe se fundamenta en:

- Precisión de detección de puntos clave corporales
- Capacidad de ejecución es dispositivos móviles
- No requiere equipamiento especializado
- Framework maduro y ampliamente validado en aplicaciones deportivas

PROMETEO

La arquitectura distribuida

Consta de Java para el backend y Python para el servicio de análisis:

- Separación de responsabilidades: lógica de negocio por un lado y procesamiento especializado por el otro
- Escalabilidad: el análisis de vídeo puede ejecutarse en instancias dedicadas
- Reutilización: el servicio de análisis puede integrarse en futuras aplicaciones

Por lo tanto, este proyecto además de cubrir una necesidad real del mercado, demuestra la aplicación práctica de tecnologías emergentes a un problema concreto del ámbito deportivo.

ANTECEDENTES

Análisis del estado del arte

El análisis biomecánico del movimiento mediante herramientas digitales es un campo en expansión, impulsado por los avances en visión artificial y el abaratamiento de los sensores de movimiento. Sin embargo, la mayoría de las soluciones disponibles presentan limitaciones relevantes en términos de coste, accesibilidad o funcionalidades, especialmente para el practicante no profesional.

Coach's Eye y Hudl Technique

Coach's Eye (TechSmith, 2024) y Hudl Technique son aplicaciones móviles orientadas al análisis cualitativo del movimiento deportivo. Permiten grabar vídeos, reproducirlos a cámara lenta y dibujar ángulos y líneas manualmente sobre la imagen. Sin embargo, el análisis es completamente manual: no existe detección automática de articulaciones ni validación frente a estándares reglamentarios. La precisión depende de la habilidad del operador.

GymAware y encoders lineales

GymAware (Kinetic Performance Technology, 2024) es el encoder lineal de referencia en el ámbito del entrenamiento basado en velocidad (VBT). Se conecta a la barra mediante un cable retráctil y mide la velocidad de desplazamiento con alta precisión, proporcionando velocidad media propulsiva, potencia y estimación del 1RM. Su principal desventaja es el coste elevado, entre 500 y 1.500 euros, y la necesidad de instalar hardware adicional en cada sesión.

Kentai y Kaia Health

Kentai y Kaia Health (2024) utilizan visión artificial para guiar ejercicios de rehabilitación y fitness en tiempo real, detectando errores de postura mediante la cámara del móvil. Sin embargo, ninguna implementa los estándares del powerlifting ni calcula la velocidad concéntrica de la barra o el 1RM. Están orientadas al fitness general, no al rendimiento en fuerza.

Tabla comparativa de soluciones

La siguiente tabla resume las funcionalidades clave de las soluciones analizadas en comparación con PerfectSquat:

Funcionalidad	GymAware/Coach's Eye		Kentai	PerfectSquat
Validación profundidad IPF automática	No	Manual	No	Sí
Velocidad concéntrica de la barra	Sí	No	No	Sí
Estimación del 1RM	Sí (sensor)	No	No	Sí (Epley)
Índice de fatiga por serie	Sí	No	No	Sí
Conteo automático de repeticiones	Sí	No	Parcial	Sí
Vídeo anotado con esqueleto	No	Manual	Sí (directo)	Sí (diferido)
Sin hardware adicional	No (>500 EUR)	Sí	Sí	Sí
Acceso gratuito	No	Freemium	Freemium	Sí
App Android nativa	No	Sí	Sí	Sí
Estándar IPF implementado	No	No	No	Sí

Como se observa en la tabla, PerfectSquat es la única solución que combina la validación automática de profundidad IPF, el cálculo de

PROMETEO

velocidad concéntrica, la estimación del 1RM, el índice de fatiga y el conteo automático de repeticiones en una aplicación móvil gratuita sin hardware adicional. Esta combinación constituye la ventaja diferencial del proyecto frente a las alternativas existentes.

INTRODUCCIÓN

Descripción general

Este proyecto consiste en el desarrollo de una aplicación móvil Android que permite a usuarios de powerlifting y entrenamiento de fuerza analizar la técnica de sus sentadillas mediante visión artificial. El sistema procesa vídeos y proporciona métricas objetivas basadas en el análisis biomecánico del movimiento.

La aplicación se compone de tres elementos principales:

- **Aplicación móvil Android** (Kotlin): interfaz de usuario para carga de vídeos y visualización de resultados
- **Backend REST** (Java/Spring Boot): gestión de usuarios, autenticación, almacenamiento de datos y orquestación del análisis
- **Servicio de análisis** (Python/MediaPipe): procesamiento de vídeo mediante visión artificial y cálculo de métricas biomecánicas

Problemas que resuelve

Falta de feedback técnico objetivo: Los deportistas que entrenan sin supervisión no pueden evaluar con precisión su rendimiento. La aplicación proporciona métricas cuantificables en lugar de apreciaciones subjetivas.

Incertidumbre sobre validez competitiva: Muchos practicantes desconocen si sus repeticiones serían consideradas válidas bajo el

PROMETEO

reglamento de la IPF (International Powerlifting Federation). El sistema valida automáticamente la profundidad según estos estándares.

Dificultad para estimar cargas de trabajo: Sin herramientas especializadas es difícil planificar entrenamientos. La estimación del 1RM basada en velocidad permite dosificar las cargas de forma más precisa.

Alcance del proyecto

Incluye:

- Análisis de sentadilla trasera (back squat) con barra libre
- Validación según estándares IPF de powerlifting
- Aplicación móvil para Android (API 26+)
- Procesamiento de vídeos pregrabados

No incluye:

- Análisis de otros ejercicios (peso muerto, press banca)
- Variantes de sentadilla (frontal, overhead, búlgara)
- Procesamiento en tiempo real durante la ejecución
- Versión para iOS o aplicación web
- Reconocimiento automático de carga levantada

OBJETIVOS

Los objetivos del proyecto se desglosan mediante el esquema RFTP (Requisito - Función - Tarea - Prueba) en cuatro bloques funcionales principales, directamente trazables con el código del repositorio.

1. RFTP: Gestión del Ciclo de Vida de la Sesión de Análisis

R - Requisito: El sistema debe gestionar sesiones de análisis con un ciclo de vida definido, desde la creación hasta la finalización, incluyendo los estados intermedios de subida de video y procesamiento.

F - Funciones:

- F1. Creación de sesión: el endpoint POST /session genera un nuevo objeto AnalysisSession con estado WAITING y timestamp de creación via AnalysisSessionService.createSession().
- F2. Gestión de estados: el enum AnalysisSessionState define seis estados: WAITING, UPLOADINGVIDEO, VIDEOUPLOADED, PROCESSING, FINISHED y ERROR. La transición se realiza en VideoService.saveVideo() y AnalysisSessionService.updateSessionWithResults().
- F3. Consulta de sesión: el endpoint GET /session/{id} permite consultar el estado y resultados de una sesión en cualquier momento.

T - Tareas:

1. T1.1. Diseñar la entidad AnalysisSession con los campos id (Long, autogenerated), state (enum), createdAt (LocalDateTime), weightKg (Integer), totalReps, validReps, estimatedRM, fatigueIndex, avgConcentricVelocity.
2. T1.2. Implementar AnalysisSessionService con los métodos createSession(), getSessionById(Long id) y updateSessionWithResults(Long id, PythonAnalysisResponse response).

PROMETEO

3. T1.3. Implementar la excepción personalizada

AnalysisSessionNotFoundException lanzada cuando no existe sesión con el id solicitado.

P - Pruebas:

- P1. Llamar a POST /session y verificar en la consola H2 (/h2-console) que se crea un registro con state = 'WAITING'.
- P2. Llamar a GET /session/{id} con un id inexistente y verificar que se devuelve 404 con mensaje de error.
- P3. Completar un análisis exitoso y verificar que el estado final es FINISHED y los campos de resultado están rellenos.

2. RFTP: Subida de Video y Orquestación del Análisis

R - Requisito: El sistema debe permitir subir un video MP4 asociado a una sesión, almacenarlo localmente, llamar de forma síncrona al servicio Python para su análisis y persistir los resultados.

F - Funciones:

- F1. Subida de video: el endpoint POST /session/{id}/video acepta un MultipartFile con el video y el parámetro opcional weightKg. Lo delega a VideoService.saveVideo().
- F2. Almacenamiento: StorageService persiste el archivo en disco y devuelve la ruta local. La entidad Video almacena filePath, processedFilePath (ruta del video anotado), uploadedAt y la relación OneToOne con la sesión.
- F3. Llamada al servicio Python: AnalysisClient.analyzeVideo() construye un PythonAnalysisRequest (sessionId, videoFilePath, weightKg) y realiza un POST a \${python.service.url}/analyze via RestTemplate. La URL del servicio se configura en application.properties con la clave python.service.url=http://localhost:8000.
- F4. Persistencia de resultados: AnalysisSessionService.updateSessionWithResults() mapea el

PROMETEO

PythonAnalysisResponse a los campos de AnalysisSession y crea las entidades RepDetail asociadas.

T - Tareas:

4. T2.1. Implementar StorageService para guardar el MultipartFile en el sistema de ficheros local del servidor.
5. T2.2. Implementar AnalysisClient como @Component de Spring con inyección de RestTemplate y la URL del servicio Python via @Value.
6. T2.3. Implementar el mapeo de PythonRepDetail a RepDetail en updateSessionWithResults(), incluyendo la asociación de cada RepDetail con su sesion padre.
7. T2.4. Gestionar el caso de error: si la llamada Python falla, establecer el estado de la sesión a ERROR y relanzar la excepción como RuntimeException.

P - Pruebas:

- P1. Subir un video de prueba de y verificar que el archivo aparece en la carpeta de almacenamiento del servidor y que la sesión pasa a estado PROCESSING.
- P2. Verificar que, tras el análisis, los registros de RepDetail en la tabla rep_detail de H2 contienen los valores de angulo y velocidad correctos.
- P3. Simular un error en el servicio Python (parandolo) y verificar que la sesión queda en estado ERROR.

3. RFTP: Análisis Biomecánico con Visión Artificial (Python)

R - Requisito: El servicio de análisis debe procesar el video recibido fotograma a fotograma, detectar los landmarks corporales, calcular ángulos articulares, detectar repeticiones y calcular las métricas de la sesión.

PROMETEO

F - Funciones:

- F1. Configuración del servidor: FastAPI expone el endpoint POST /analyze que instancia SquatAnalyzer con session_id, video_path y weight_kg, y llama a analyzer.analyze().
- F2. Lectura de video: cv2.VideoCapture lee el video frame a frame. Se obtiene el FPS real del video (cap.get(cv2.CAP_PROP_FPS)); si es 0 o NaN, se usa 30.0 como valor por defecto.
- F3. Detección dinámica de perfil: en cada fotograma se calcula la visibilidad media de cadera, rodilla y tobillo para el lado izquierdo y el derecho. Se elige el lado con mayor visibilidad media. Se descarta el fotograma si la visibilidad máxima es inferior a 0.3.
- F4. Calculo de ángulos: el método calculate_angle(a, b, c) calcula el angulo en el vértice b usando np.arctan2 sobre los vectores b->a y b->c, convirtiendo a grados y ajustando a [0, 180].
- F5. Maquina de estados para detección de repeticiones: STANDING (angulo rodilla > 160) -> DESCENDING (angulo rodilla < 150) -> ASCENDING (angulo rodilla sube mas de 10 grados sobre el mínimo) -> STANDING. Al volver a STANDING se cierra la repetición.
- F6. Validación de profundidad: una repetición es valida si current_min_knee_angle <= 90.0 grados durante la fase de descenso.
- F7. Calculo de velocidad concéntrica: velocidad = 1.0 / (frames_en_ascenso / fps). Representa la velocidad de subida en unidades normalizadas por segundo.
- F8. Generación de video anotado: el video de salida se genera con sufijo _analyzed.mp4, con el esqueleto MediaPipe superpuesto, el angulo de rodilla en pantalla, el estado actual (STATE) y el contador de repeticiones validas/totales (REPS).

T - Tareas:

8. T3.1. Configurar el entorno Python con las dependencias de requirements.txt: fastapi==0.109.2, uvicorn==0.27.1, mediapipe==0.10.14, opencv-python, pydantic==2.6.1.

PROMETEO

9. T3.2. Implementar la clase SquatAnalyzer con el metodo analyze() que encapsula todo el procesamiento de video.
10. T3.3. Implementar los modelos Pydantic: PythonAnalysisRequest (sessionId, videoFilePath, weightKg), PythonRepDetail (repNumber, valid, minHipAngle, minKneeAngle, concentricVelocity) y PythonAnalysisResponse (sessionId, processedVideoFilePath, totalReps, validReps, reps, averageConcentricVelocity, estimatedRM, fatigueIndex).
11. T3.4. Iniciar el servidor con uvicorn: uvicorn main:app --host 0.0.0.0 --port 8000 --reload.

P - Pruebas:

- P1. Enviar un video con 5 repeticiones todas validas y verificar que la respuesta devuelve totalReps=5, validReps=5.
- P2. Enviar un video con 3 repeticiones validas y 2 sin llegar a 90 grados y verificar el conteo correcto.
- P3. Verificar que el video de salida _analyzed.mp4 contiene el esqueleto superpuesto y los textos de estado visibles.

4. RFTP: Interfaz de Usuario Android (Jetpack Compose)

R - Requisito: La aplicación Android debe ofrecer una interfaz funcional con cinco pantallas (menú, inicio de análisis, resultados, histórico y estadísticas), navegación fluida y comunicación con el backend via Retrofit.

F - Funciones:

- F1. Punto de entrada: MainActivity establece el tema FEPerfectSquatTheme y llama a AppNavigation(), que configura el NavHost con cinco destinos composables.
- F2. MenuScreen: pantalla central con cuatro botones de navegación ('Iniciar análisis', 'Histórico sentadillas', 'Estadísticas', 'Salir') y titulo 'PerfectSquat'.

PROMETEO

- F3. StartAnalysisScreen: campo de texto numérico para el peso (OutlinedTextField), selector de video vía `ActivityResultContracts.GetContent()` filtrando 'video/mp4', y botón 'Analizar' que llama al backend.
- F4. AnalysisResultScreen: muestra el video analizado y tres botones: Guardar, Borrar y Repetir.
- F5. HistoryScreen: lista de sesiones previas en LazyColumn con Cards.
- F6. StatisticsScreen: Cards con total de repeticiones históricas y RM histórico, con espacio para gráficas futuras.
- F7. Cliente de red: RetrofitClient configura Retrofit con base URL `http://10.0.2.2:8080/api/` (direccion del emulador Android hacia localhost) y conversor GsonConverterFactory. ApiService define los endpoints GET `session/{id}` y POST `session/{id}/video`.

DESCRIPCIÓN

Arquitectura del sistema

PerfectSquat sigue una arquitectura de tres capas con comunicación HTTP REST sincrónica. El backend Java actúa como orquestador central: recibe las peticiones del cliente móvil, gestiona la persistencia y delega el procesamiento de video al servicio Python.

En la siguiente tabla muestro las tecnologías utilizadas por paquete y sus responsabilidades:

PROMETEO

Paquete	Tecnologia	Responsabilidad principal
FEPerfectSquat	Kotlin 2.2.10 / Jetpack Compose / Material3	UI, navegación, selección de video, visualización de resultados
BEPerfectSquat	Java 17 / Spring Boot 3.4.13 / H2	Gestión de sesiones, almacenamiento de videos, orquestacion
PYAnalysisService	Python 3.11 / FastAPI / MediaPipe 0.10.14	Procesamiento de video, detección de pose, cálculos biomecánicos

Flujo de datos detallado

El flujo de datos completo sigue estos pasos numerados, trazables directamente con el código:

1. La app Android llama a POST /session (AnalysisSessionController.createSession()). Se crea un AnalysisSession con state=WAITING y se devuelve el id de sesión.
2. La app llama a POST /session/{id}/video con el archivo MP4 como multipart y el peso como parametro (VideoService.saveVideo()). El backend cambia el estado a PROCESSING.
3. StorageService guarda el archivo en disco y devuelve la ruta local (localFilePath).
4. AnalysisClient.analyzeVideo() construye un PythonAnalysisRequest y realiza POST http://localhost:8000/analyze.
5. FastAPI recibe la petición en el endpoint /analyze, instancia SquatAnalyzer y llama a analyzer.analyze().
6. SquatAnalyzer procesa el video con MediaPipe, detecta landmarks frame a frame y aplica la maquina de estados para contar repeticiones. Genera el video anotado con sufijo _analyzed.mp4.

PROMETEO

7. PythonAnalysisResponse (totalReps, validReps, reps[], averageConcentricVelocity, estimatedRM, fatigueIndex, processedVideoFilePath) se devuelve al backend.
8. AnalysisSessionService.updateSessionWithResults() mapea la respuesta a la entidad AnalysisSession, crea los RepDetail y guarda en H2. El estado pasa a FINISHED.
9. La app consulta GET /session/{id} y muestra los resultados en AnalysisResultScreen.

A continuación muestro un diagrama con el flujo de datos resumido:

PROMETEO



Descripción de los módulos

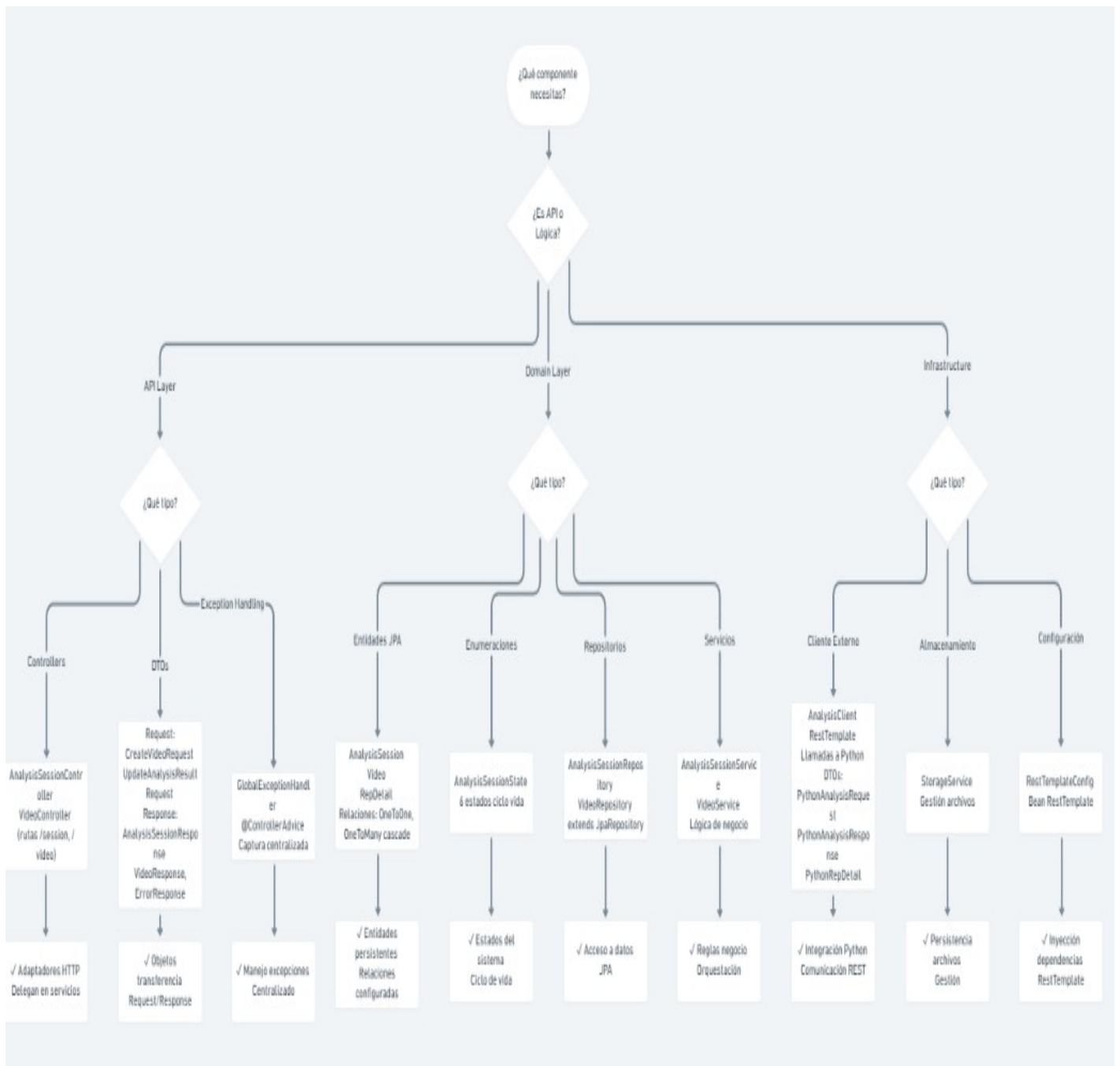
BEPerfectSquat: estructura de paquetes

El backend sigue una estructura de capas inspirada en arquitectura hexagonal:

- api/controller: AnalysisSessionController (rutas /session) y VideoController (rutas /video). Adaptadores HTTP que delegan en los servicios de dominio.
- api/dto: objetos de transferencia para peticiones (CreateVideoRequest, UpdateAnalysisResultRequest) y respuestas (AnalysisSessionResponse, VideoResponse, ErrorResponse).
- api/exception: GlobalExceptionHandler con @ControllerAdvice para captura centralizada de excepciones.
- domain/entity: AnalysisSession, Video y RepDetail como entidades JPA. AnalysisSession tiene relacion OneToOne con Video y OneToMany con RepDetail (cascade ALL, orphanRemoval).
- domain/enums: AnalysisSessionState con los seis estados del ciclo de vida.
- domain/repository: AnalysisSessionRepository y VideoRepository como interfaces JpaRepository.
- domain/service: AnalysisSessionService y VideoService con la lógica de negocio.
- infrastructure/client: AnalysisClient con RestTemplate para la llamada al servicio Python. PythonAnalysisRequest, PythonAnalysisResponse y PythonRepDetail como DTOs del contrato Python.
- infrastructure/service: StorageService para la gestión del almacenamiento de archivos.
- infrastructure/config: RestTemplateConfig para la instancia del bean RestTemplate.

PROMETEO

Aquí muestro un diagrama que define la estructura de paquetes del backend según componente:



PROMETEO

PYAnalysisService: lógica de análisis

El servicio Python se organiza en tres archivos:

- main.py: punto de entrada FastAPI. Define el endpoint POST /analyze, instancia SquatAnalyzer y gestiona el logging y el manejo de excepciones con HTTPException(500).
- models.py: modelos Pydantic para validación de entrada y salida. PythonAnalysisRequest, PythonRepDetail y PythonAnalysisResponse.
- services/squat_analyzer.py: clase SquatAnalyzer con toda la lógica de procesamiento de video.

El algoritmo de detección de repeticiones en SquatAnalyzer implementa una maquina de tres estados:

Estado	Condición de entrada	Condición de salida	Acción
STANDING	Ángulo rodilla > 160	Ángulo rodilla < 150	Estado inicial y final de cada rep
DESCENDING	Ángulo rodilla < 150 desde STANDING	Ángulo rodilla sube > 10 sobre el mínimo	Se registra min_knee_angle y min_hip_angle
ASCENDING	Ángulo sube > 10 sobre mínimo en DESCENDING	Ángulo rodilla > 160	Se cuenta frames_in_ascent para velocidad

Casos de uso

CU-01: Iniciar análisis de video de sentadilla

DESCRIPCIÓN: El usuario selecciona un video MP4 de sentadilla, introduce el peso utilizado y lanza el análisis mecanicismo automático.

PRECONDICIONES

El usuario tiene la app instalada.
Existe al menos un video MP4 de sentadilla grabado en la galería del dispositivo.

DATOS ENTRADA

Video MP4 (MultipartFile)
weightKg (Double, opcional)
sessionId (generado en POST /session)

TABLAS

analysis_session
video
rep_detail

INTERFACES

StartAnalysisScreen.kt
AnalysisResultScreen.kt

POSTCONDICIONES

Se crea una sesión con state=FINISHED en la BD.
Los resultados (repeticiones, velocidades, 1RM, fatiga) están disponibles para consulta.
El video anotado con el esqueleto se almacena en el servidor.

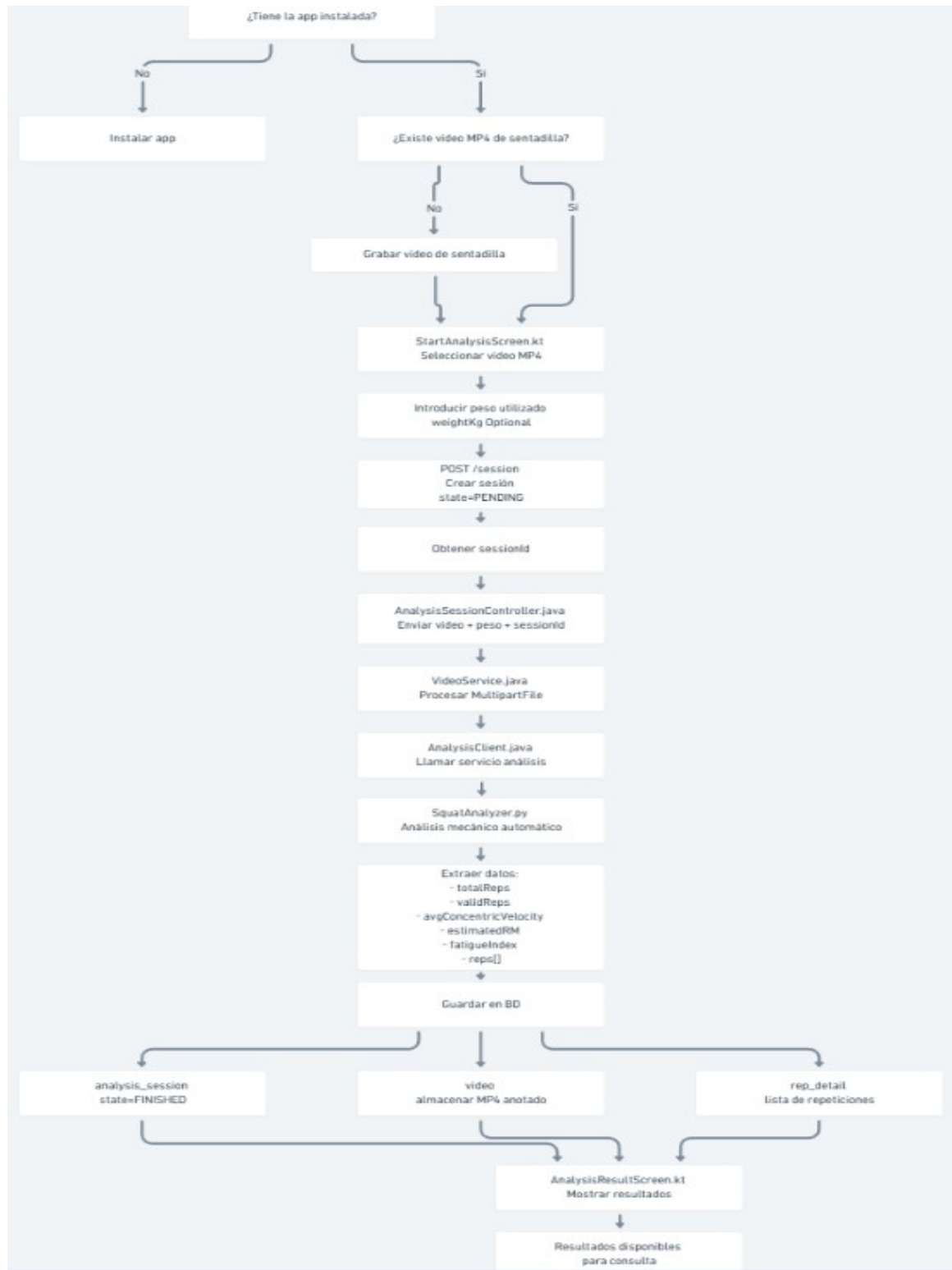
DATOS SALIDA

id de sesion (Long)
totalReps (Integer)
validReps (Integer)
avgConcentricVelocity (Double)
estimatedRM (Double)
fatigueIndex (Double)
reps[] (lista de RepDetail)

CLASES

AnalysisSessionController.java
VideoService.java
AnalysisClient.java
SquatAnalyzer.py

PROMETEO



CU-02: Consultar historial de sesiones

DESCRIPCIÓN: El usuario navega al histórico para ver el listado de sesiones de análisis previas.

PRECONDICIONES

Existe al menos una sesión con state=FINISHED en la base de datos.

DATOS ENTRADA

(ninguno — solo navegación desde MenuScreen)

TABLAS

analysis_session

INTERFACES

HistoryScreen.kt

AppNavigation.kt (destino history)

POSTCONDICIONES

Se muestra el listado de sesiones en HistoryScreen con los datos de cada una.

DATOS SALIDA

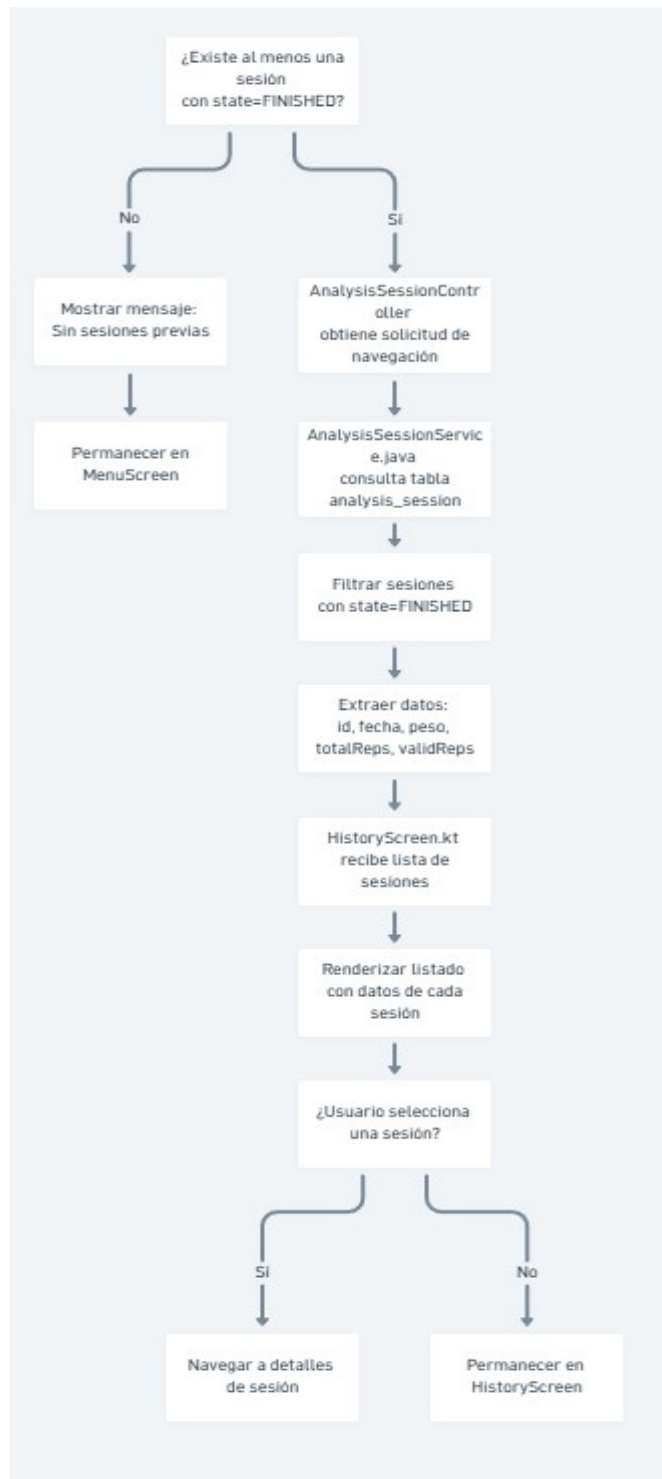
Lista de sesiones con id, fecha, peso, totalReps, validReps

CLASES

AnalysisSessionController.java

AnalysisSessionService.java

PROMETEO



CU-03: Ver estadísticas globales

DESCRIPCIÓN: El usuario accede a la pantalla de estadísticas para ver el total de repeticiones históricas y el RM máximo registrado.

PRECONDICIONES

El usuario ha realizado al menos una sesión de análisis finalizada.

DATOS ENTRADA

(ninguno — agregación de sesiones existentes)

TABLAS

analysis_session

INTERFACES

StatisticsScreen.kt

AppNavigation.kt (destino statistics)

POSTCONDICIONES

Se muestra StatisticsScreen con los indicadores de total de repeticiones y RM histórico.

DATOS SALIDA

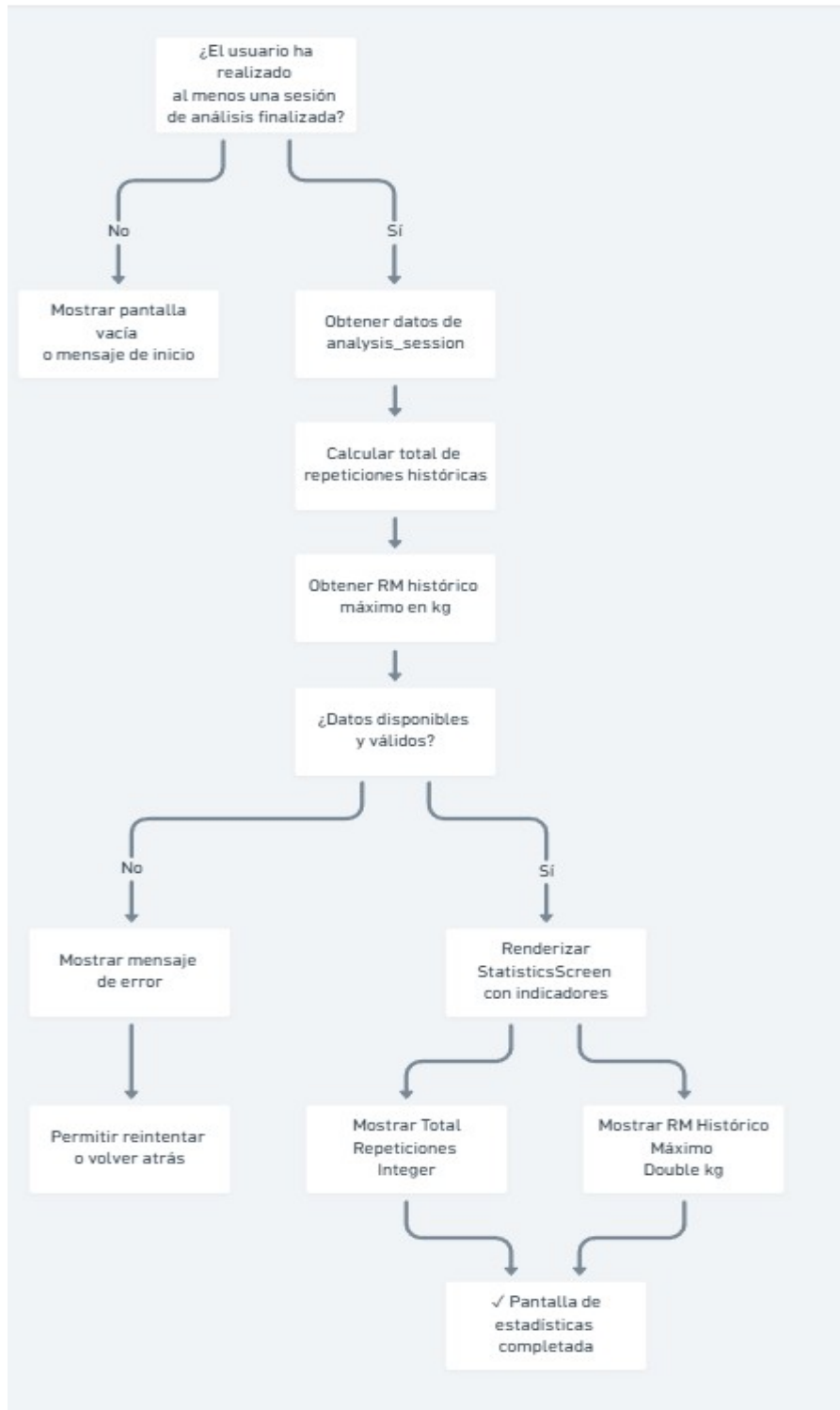
Total repeticiones históricas (Integer)

RM histórico máximo (Double, en kg)

CLASES

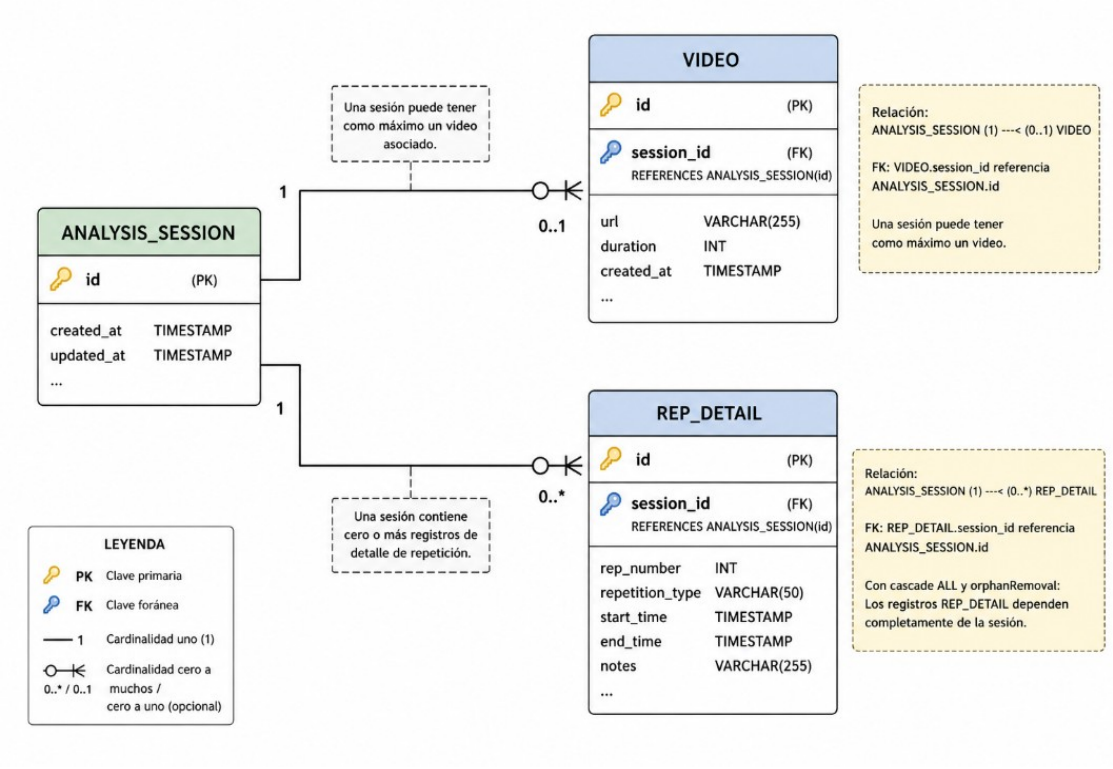
AnalysisSessionService.java

PROMETEO



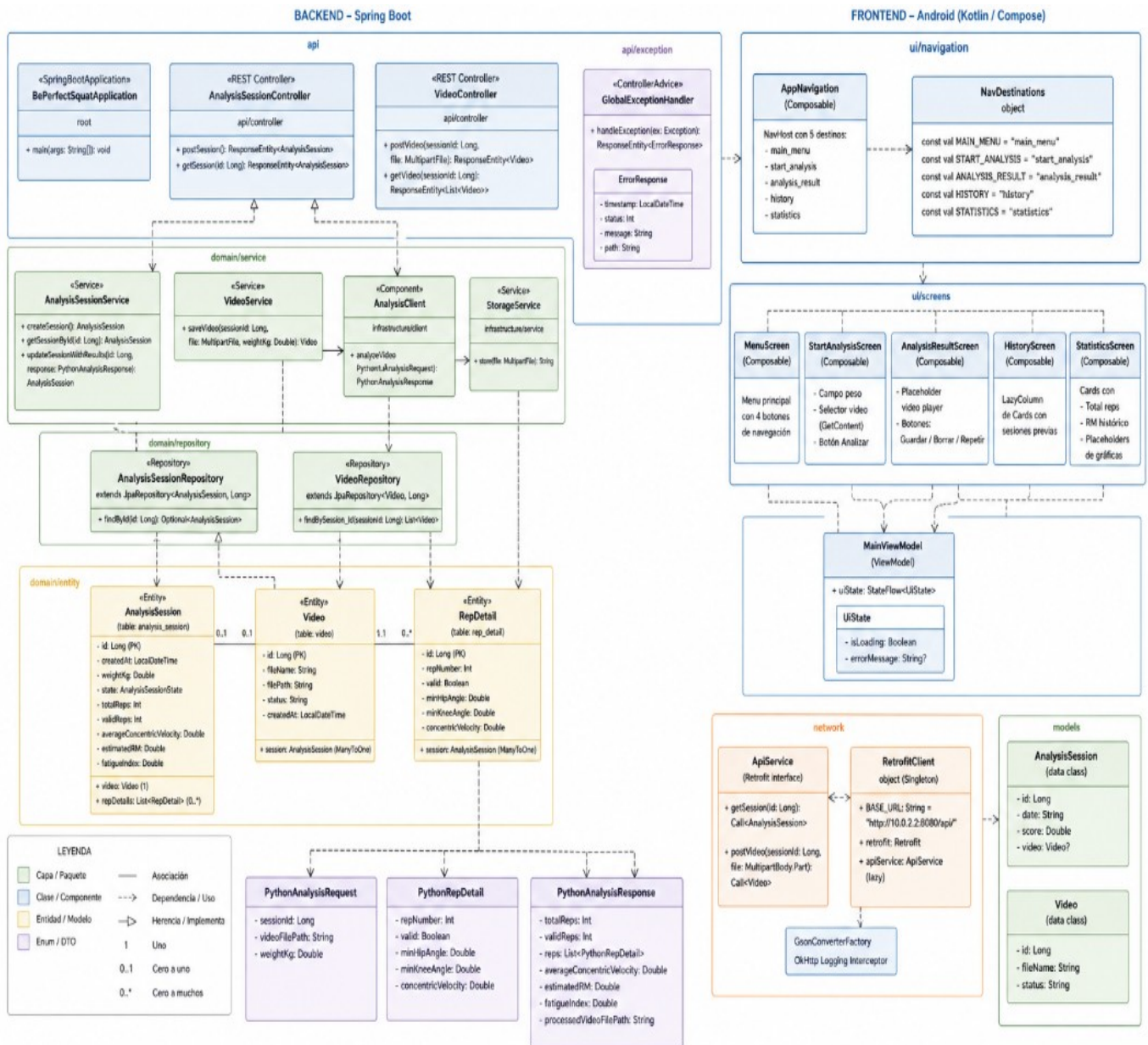
DISEÑOS

Diagrama Entidad-Relacion (E/R)



PROMETEO

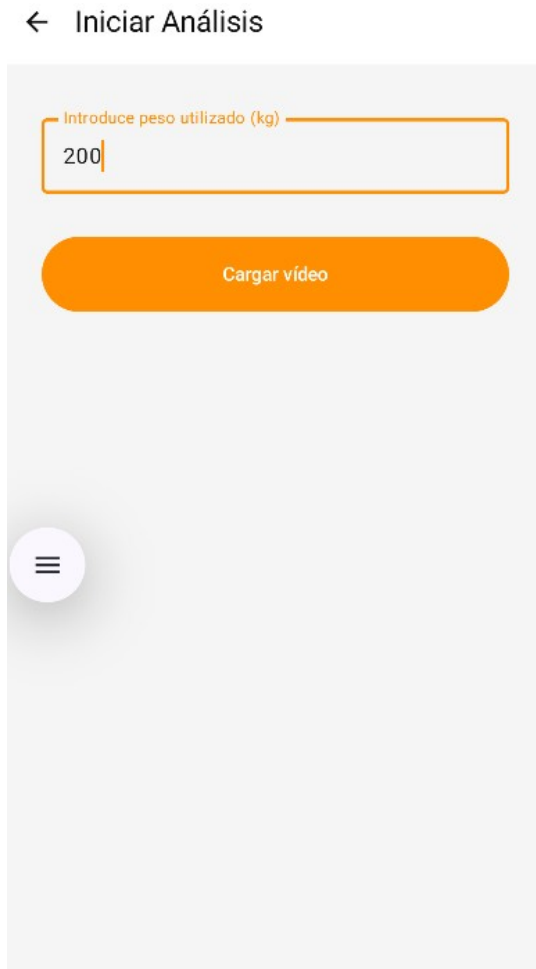
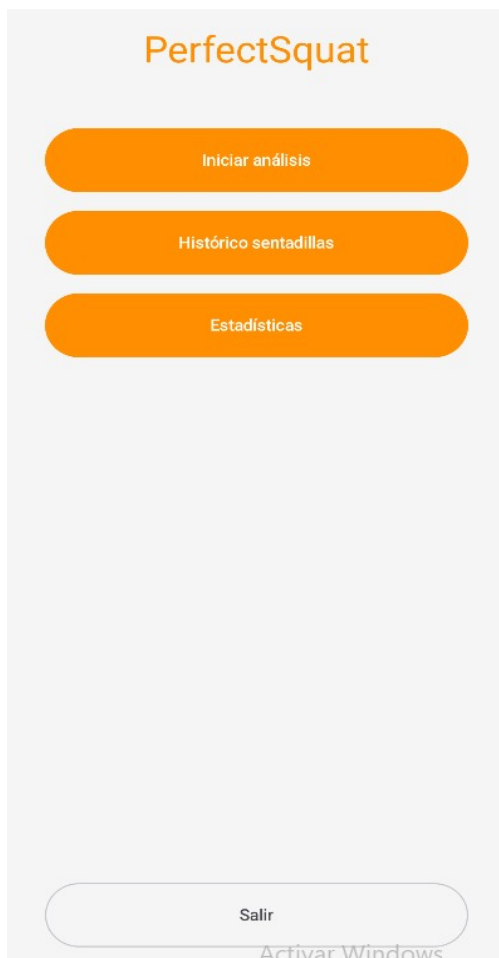
Diagrama de clases – Backend/Frontend



PROMETEO

Diseño de interfaces — Pantallas reales

Pantallas menús



PROMETEO

← Histórico

Video 1

Video 2

Video 3

Video 4

Video 5

Video 6

Video 7

Video 8

Video 9

Video 10

← Estadísticas

Total repes

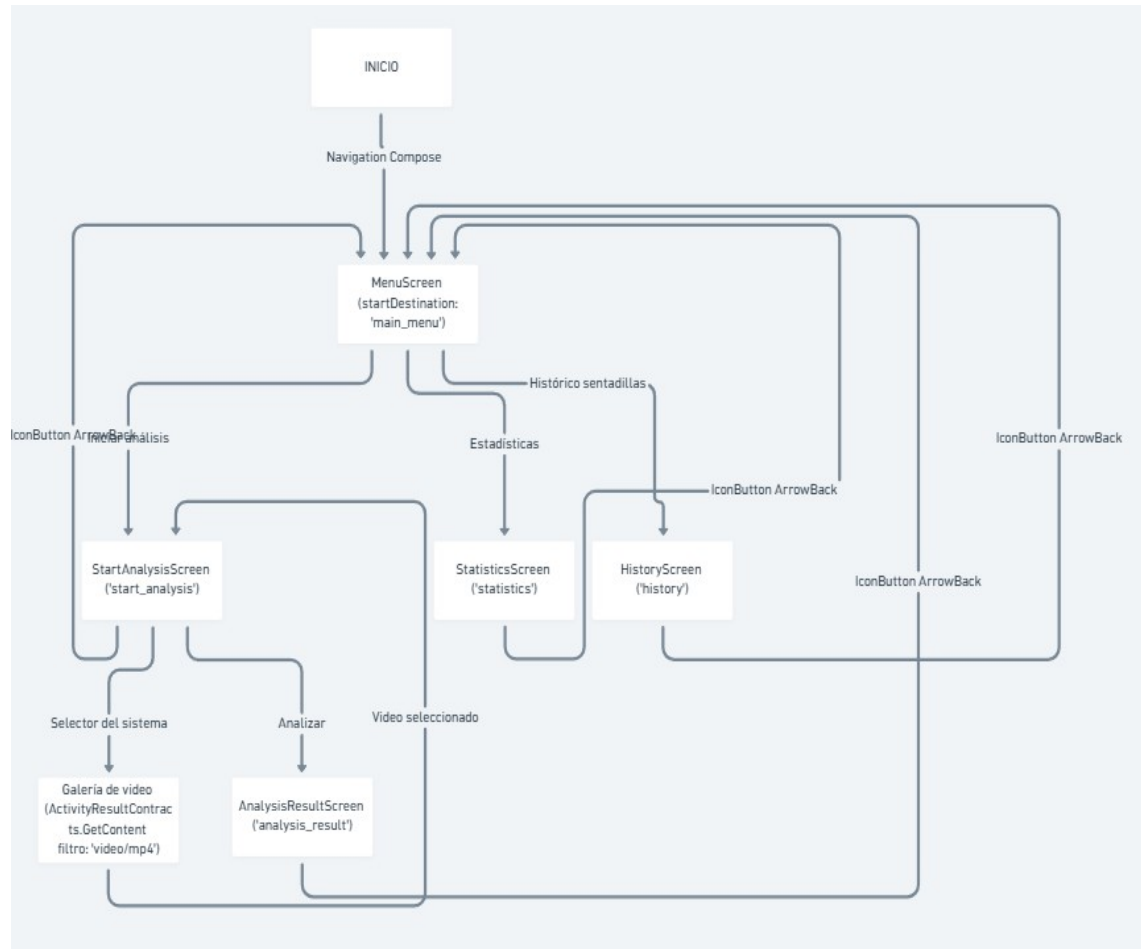
0

RM histórico

0 kg

PROMETEO

Diagrama de flujo de navegación



TECNOLOGÍA



Java (Spring Boot)

Descripción: Lenguaje de programación de propósito general, orientado a objetos y de tipado fuerte. Se utiliza el framework Spring Boot para la creación de microservicios robustos y escalables.

Uso en el proyecto: Actúa como el cerebro del backend. Gestiona la lógica de negocio, el ciclo de vida de las sesiones de entrenamiento, la persistencia en base de datos y la orquestación de llamadas al servicio de análisis.



Python (MediaPipe & OpenCV)

Descripción: Lenguaje interpretado de alto nivel, líder en el sector de la Inteligencia Artificial y Visión Artificial debido a su ecosistema de librerías científicas.

Uso en el proyecto: Es el motor de análisis biomecánico. Recibe los archivos de vídeo, utiliza la librería MediaPipe para extraer los puntos clave del cuerpo y calcula mediante algoritmos la velocidad y profundidad de la sentadilla.

Kotlin

Descripción: Lenguaje moderno, conciso y seguro, designado por Google como el lenguaje preferido para el desarrollo nativo en Android.



Uso en el proyecto: Desarrollo de la interfaz de usuario. Se encarga de la interacción con el usuario, la selección de vídeos de la galería mediante la API de Android y la visualización de los resultados del análisis en gráficas e indicadores.



PostgreSQL

Descripción: Sistema de gestión de bases de datos relacional de código abierto, conocido por su fiabilidad, integridad de datos y soporte para consultas complejas.

Uso en el proyecto: Almacenamiento persistente de datos. Guarda la información de las sesiones, las rutas de los archivos de vídeo y, lo más importante, las métricas de cada repetición (velocidad, fecha, estado) para que el usuario pueda consultar su histórico.

Descripción de tecnologías clave

MediaPipe Pose

MediaPipe Pose es el framework de estimación de pose utilizado en el servicio Python. Se inicializa con `min_detection_confidence=0.5` y `min_tracking_confidence=0.5`. Devuelve 33 landmarks normalizados con coordenadas (x, y, z) y un valor de visibilidad. Los landmarks relevantes para el análisis de sentadilla son: RIGHT_HIP (24) / LEFT_HIP (23), RIGHT_KNEE (26) / LEFT_KNEE (25), RIGHT_ANKLE (28) / LEFT_ANKLE (27) y RIGHT_SHOULDER (12) / LEFT_SHOULDER (11). La selección entre lado derecho e izquierdo se realiza dinámicamente en cada fotograma comparando la visibilidad media de los tres puntos de cada extremidad, garantizando un análisis correcto independientemente del lado de grabación.

Algoritmo de calculo de angulo

El método `calculate_angle(a, b, c)` calcula el angulo en el vértice b formado por los tres puntos a, b, c mediante la formula:

$$\text{radians} = \arctan2(c[1]-b[1], c[0]-b[0]) - \arctan2(a[1]-b[1], a[0]-b[0])$$

PROMETEO

```
angle = abs(radians * 180 / pi)
```

```
if angle > 180: angle = 360 - angle
```

Para el angulo de rodilla, los puntos son: a=cadera, b=rodilla, c=tobillo.

Para el angulo de cadera: a=hombro, b=cadera, c=rodilla. El resultado esta en el rango [0, 180] grados.

Formula de estimación del 1RM (Epley)

La estimación del 1RM se calcula mediante la formula de Epley, implementada directamente en `squat_analyzer.py`:

```
est_rm = weight_kg * (1 + 0.0333 * valid_reps)
```

Esta formula devuelve el peso máximo estimado que el usuario podría levantar en una sola repetición, extrapolado a partir del peso utilizado y el numero de repeticiones validas realizadas en la serie. Solo se calcula si `weight_kg` fue proporcionado en la petición.

Calculo del indice de fatiga

El indice de fatiga se calcula como la caída de velocidad concéntrica entre la primera y la ultima repetición de la serie:

```
fatigue = 1 - (last_velocity / first_velocity)
```

Un valor de 0 indica sin fatiga (velocidades constantes). Un valor de 0.2 indica una caída del 20% en la velocidad entre la primera y la ultima repetición. El indice no se calcula si hay menos de dos repeticiones en la serie.

Configuración del backend (`application.properties`)

El backend se configura con H2 en memoria para el entorno de desarrollo:

```
spring.datasource.url=jdbc:h2:mem:testdb
```

```
spring.datasource.driverClassName=org.h2.Driver
```

PROMETEO

```
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
python.service.url=http://localhost:8000
```

La consola H2 queda disponible en <http://localhost:8080/h2-console> para inspección de la base de datos en desarrollo. La URL del servicio Python es configurable mediante la propiedad `python.service.url`, lo que facilita cambiarla a un entorno de producción sin modificar el código fuente.

METODOLOGÍA

Metodología empleada

Para el desarrollo de PerfectSquat se ha utilizado una metodología ágil de desarrollo incremental basada en sprints cortos de dos semanas. Al tratarse de un proyecto individual, las ceremonias de Scrum se han adaptado: se sustituyen las reuniones diarias por un registro personal de tareas y se mantienen las retrospectivas al final de cada sprint para identificar mejoras en el proceso.

La elección de esta metodología se justifica por la naturaleza exploratoria del trabajo con visión artificial: los umbrales del algoritmo de detección de repeticiones (ángulo de inicio de bajada < 150 , ángulo de pie > 160 , delta de ascenso > 10 grados) requirieron varios ciclos de ajuste experimental sobre videos reales antes de alcanzar un comportamiento satisfactorio. Un enfoque en cascada habría dificultado notablemente este proceso iterativo.

El repositorio de código se gestiona con Git y GitHub . La estructura de ramas sigue el flujo main / feature-branches, con commits frecuentes que documentan el progreso de cada tarea.

PROMETEO

Planificación inicial estimada

Sprint	Modulo	Descripción	Horas estimadas
S1	Planificación	Documentación inicial, definición de requisitos, diseño de arquitectura	20
S2	Backend	Setup Spring Boot 3.4, entidades JPA, repositorios, H2, endpoints básicos	25
S3	Backend	VideoService, StorageService, AnalysisClient, integración con Python	25
S4	Python	Setup FastAPI, modelos Pydantic, lectura de video con OpenCV	20
S5	Python	Integración MediaPipe, detección de landmarks, calculo de ángulos	30
S6	Python	Maquina de estados, detección de repeticiones, validación de profundidad	30
S7	Python	Calculo de velocidad, 1RM Epley, fatiga, generación de video anotado	20
S8	Android	Setup Compose, tema Material3, navegación AppNavigation, 5 pantallas base	25
S9	Android	Integracion Retrofit, ApiService, RetrofitClient, subida de video multipart	25
S10	Android	ViewModel, StateFlow, conexión con resultados del backend	20
S11	Integración	Pruebas end-to-end, ajuste de umbrales, corrección de errores	30
S12	Documentación	Memoria TFG, README, diagramas finales	25
TOTAL ESTIMADO			295

Planificación real ejecutada

Sprint	Modulo	Horas estimadas	Horas reales	Desviación
S1	Planificación	20	22	2
S2	Backend	25	27	2
S3	Backend	25	28	3
S4	Python	20	24	4
S5	Python	30	36	6
S6	Python	30	40	10
S7	Python	20	22	2
S8	Android	25	28	3
S9	Android	25	30	5
S10	Android	20	22	2
S11	Integración	30	35	5
S12	Documentación	25	28	3
	TOTAL	295	342	47

La desviación total fue de 47 horas (un 16% sobre la estimación). La mayor desviación se produjo en el Sprint 6 (detección de repeticiones), donde el ajuste de los umbrales de la maquina de estados requirió mas iteraciones de las previstas sobre videos reales. Los ángulos de transición (160/150 grados para STANDING/DESCENDING y +10 grados para detectar el inicio del ascenso) se determinaron empíricamente tras probar con videos de distintas velocidades de ejecución y morfologías corporales. El Sprint 5 (integracion MediaPipe) también supero la estimación por la necesidad de implementar la detección dinámica de perfil izquierdo/derecho, que no estaba contemplada en el diseño inicial.

PROMETEO

Presupuesto del proyecto

Concepto	Cantidad	Coste unitario (EUR)	Coste total (EUR)
Horas de desarrollo (desarrollador junior)	342 h	25 EUR/h	8550
Android Studio (IDE gratuito)	1	0	0
IntelliJ IDEA Community (IDE gratuito)	1	0	0
Spring Boot 3.4.13 (framework open source)	1	0	0
MediaPipe 0.10.14 (librería open source)	1	0	0
FastAPI / Uvicorn (open source)	1	0	0
H2 Database (open source)	1	0	0
GitHub	1	0	0
		TOTAL	8550

La totalidad del coste del proyecto corresponde al tiempo de desarrollo. Todos los frameworks, librerías, IDEs y servicios de control de versiones utilizados son gratuitos y de código abierto, lo que hace que el coste de licencias sea nulo.

DESPLIEGUE E INSTALACIÓN

Requisitos previos del sistema

Antes de arrancar los servicios es necesario que el equipo cumpla los siguientes requisitos mínimos:

Componente	Requisito mínimo	Descripción
Java JDK	Java 17 LTS	Compilar y ejecutar el backend Spring Boot
Maven Wrapper	Incluido en repo	Gestión de dependencias (./mvnw)
Python	3.10 o superior	Servicio de análisis PYAnalysisService
pip	23.x o superior	Instalación de dependencias Python
Android Studio	Hedgehog 2023.1+	Compilar y desplegar la app Android
Android SDK	API 26 (Android 8.0)	Nivel mínimo de la app
RAM servidor	4 GB mínimo	Necesaria para MediaPipe en CPU
Disco libre	2 GB mínimo	Almacenamiento de vídeos analizados

Instalación del servicio Python (PYAnalysisService)

1. Clonar el repositorio: git clone

<https://github.com/MIz97/VisualAnalysisSquat.git>

2. Crear entorno virtual e instalar dependencias (fastapi==0.109.2, uvicorn==0.27.1, mediapipe==0.10.14, opencv-python, pydantic==2.6.1):

```
python -m venv venv | source venv/bin/activate | pip install -r requirements.txt
```

3. Arrancar el servidor en el puerto 8000:

```
cd PYAnalysisService/app | uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

PROMETEO

El servidor quedará disponible en <http://localhost:8000>. La documentación Swagger UI está accesible en <http://localhost:8000/docs> para verificar que el servicio está operativo.

Instalación del backend Java (BEPerfectSquat)

1. Navegar al directorio: `cd VisualAnalysisSquat/BEPerfectSquat`
2. Compilar y arrancar con el Maven Wrapper incluido (no requiere Maven global):

```
./mvnw spring-boot:run (Linux/Mac) | mvnw.cmd spring-boot:run  
(Windows)
```

El backend arrancará en <http://localhost:8080>. La consola H2 para inspección de la BD está disponible en <http://localhost:8080/h2-console> (JDBC URL: `jdbc:h2:mem:testdb`, usuario: `sa`, contraseña vacía). La URL del servicio Python se configura mediante `python.service.url` en `application.properties`.

Configuración de la app Android (FEPerfectSquat)

1. Abrir el directorio FEPerfectSquat en Android Studio (File > Open).
2. Configurar la URL del backend en `RetrofitClient.kt`. Para el emulador: `BASE_URL = http://10.0.2.2:8080/api/`. Para dispositivo físico en la misma red WiFi: sustituir por la IP local del servidor.
3. Ejecutar en emulador o dispositivo: Run > Run 'app' en Android Studio, o `./gradlew installDebug` desde línea de comandos.

PROMETEO

Verificación y planes de contingencia

Orden de arranque obligatorio: (1) PYAnalysisService en el puerto 8000, (2) BEPerfectSquat en el puerto 8080, (3) app Android. Secuencia de verificación:

- POST `http://localhost:8080/session` → debe devolver JSON con `state: WAITING`.
- POST `http://localhost:8080/session/1/video` con un MP4 de prueba → debe devolver `state: FINISHED` tras el análisis.
- GET `http://localhost:8080/session/1` → debe mostrar `totalReps`, `validReps` y `estimatedRM` con valores numéricos.

En caso de fallo del servicio Python, VideoService establece el estado de la sesión a ERROR de forma controlada. Para entornos de producción, se recomienda migrar de H2 a PostgreSQL, cuyo driver ya está incluido en el `pom.xml`.

TRABAJOS FUTUROS

Mejoras a corto plazo (pendientes en el propio repositorio)

- Integración completa de la pantalla AnalysisResultScreen con los datos reales del backend: actualmente muestra un placeholder; pendiente conectar el ViewModel con el endpoint GET /session/{id} y mostrar totalReps, validReps, avgConcentricVelocity, estimatedRM y fatigueIndex.
- Implementación del historial real en HistoryScreen: actualmente muestra 10 items de prueba; pendiente implementar el endpoint GET /session (listado de todas las sesiones) y conectarlo con LazyColumn.
- Implementación de StatisticsScreen con datos reales: agregar total de repeticiones y RM máximo histórico desde el backend.
- Reproductor de video en AnalysisResultScreen: integrar ExoPlayer para reproducir el video anotado (_analyzed.mp4) generado por Python.
- Migración a PostgreSQL: el driver ya esta incluido en el pom.xml; pendiente configurar application.properties para producción con las credenciales de PostgreSQL.

Mejoras a medio plazo

- Análisis en tiempo real: implementar un modo de análisis en directo usando CameraX y MediaPipe Tasks Vision para procesar los frames de la cámara con latencia inferior a 200 ms, eliminando la necesidad de grabar y subir el video.
- Versión iOS o multiplataforma: migrar la app a Kotlin Multiplatform Mobile (KMM) o Flutter para dar soporte a usuarios de iPhone sin duplicar la lógica de negocio.
- Reconocimiento automático del peso: integrar un modulo OCR o clasificador de imágenes para detectar los discos de la barra y calcular el peso cargado sin introducción manual.

PROMETEO

- Análisis de verticalidad de la barra: implementar el seguimiento de la trayectoria del punto medio entre los hombros durante el movimiento para detectar desviaciones laterales indicativas de asimetrías musculares.
- Sistema de autenticación: implementar registro e inicio de sesión con Spring Security y JWT para sincronizar el historial de sesiones en la nube y permitir acceso desde múltiples dispositivos.

Mejoras a largo plazo

- Modelo de IA personalizado: entrenar un modelo de deep learning específico para la sentadilla con datos anotados propios, que mejore la precisión de MediaPipe en condiciones de oclusión parcial, ropa oscura o fondos complejos.
- Ampliación a otros levantamientos: extender el servicio Python para analizar el peso muerto y el press de banca, completando la triada del powerlifting.
- Panel web para entrenadores: desarrollar un dashboard web con gráficas históricas para que entrenadores puedan monitorizar el progreso de múltiples atletas.
- Predicción de lesiones: analizar longitudinalmente patrones de movimiento de riesgo (valgo de rodilla, inclinación excesiva del tronco) y generar alertas preventivas.

CONCLUSIONES

El proyecto PerfectSquat ha demostrado la viabilidad técnica de un sistema de análisis biomecánico automatizado de la sentadilla ejecutable en un entorno de tres capas ligero, sin necesidad de equipamiento adicional ni procesamiento en tiempo real.

Desde el punto de vista técnico, la integración de MediaPipe 0.10.14 con FastAPI ha resultado ser la combinación mas eficiente para el servicio de análisis: MediaPipe proporciona detección de landmarks suficientemente precisa para calcular ángulos articulares fiables, mientras que FastAPI permite exponer el servicio de forma rápida y con validación automática de esquemas mediante Pydantic. La maquina de estados implementada en SquatAnalyzer (STANDING -> DESCENDING -> ASCENDING) ha demostrado ser un patrón robusto para la detección de ciclos de repetición, aunque los umbrales de angulo requieren calibración empírica sobre videos reales.

El uso de Spring Boot 3.4.13 como orquestador central ha permitido una separación limpia de responsabilidades: el backend no contiene lógica de visión artificial y el servicio Python no gestiona persistencia. Esta arquitectura facilita el mantenimiento y el escalado independiente de cada capa. El uso de H2 en memoria simplifica enormemente el entorno de desarrollo al eliminar la necesidad de instalar y configurar un servidor de base de datos externo.

La elección de Jetpack Compose con Material3 para la interfaz Android ha acelerado el desarrollo de las cinco pantallas de la aplicación. El sistema de navegación con NavHost y rutas composables (AppNavigation) resulta mas mantenible que el sistema tradicional de Actividades e Intents, especialmente en una aplicación de pantalla única como esta.

En cuanto a los objetivos iniciales, se han implementado todos los módulos del sistema y el flujo completo de análisis funciona de extremo a extremo. Las funcionalidades de visualización de resultados en la app

PROMETEO

Android (AnalysisResultScreen, HistoryScreen, StatisticsScreen) están estructuradas pero pendientes de conectar con los datos reales del backend, lo que constituye el siguiente paso inmediato de desarrollo.

Desde el punto de vista formativo, este proyecto ha integrado de forma práctica competencias transversales de todo el ciclo: desarrollo de aplicaciones móviles nativas, arquitectura de APIs REST, diseño de bases de datos, contenedores de servicios, programación orientada a objetos en tres lenguajes (Kotlin, Java, Python) y, de forma autodidacta, visión por computador y biomecánica deportiva. La combinación de todas estas disciplinas en un único proyecto cohesionado representa el principal valor añadido de este trabajo de fin de grado.

REFERENCIAS

Vision artificial y MediaPipe

Aplicado en la implementacion de squat_analyzer.py (deteccion de pose y calculo de angulos).

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., y Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. arXiv preprint arXiv:1906.08172. <https://arxiv.org/abs/1906.08172>

Aplicado en el procesamiento fotograma a fotograma con cv2.VideoCapture y la generacion del video anotado.

Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. Recuperado de <https://opencv.org/>

Biomecanica y formula de estimacion del 1RM

Aplicado en el calculo del 1RM con formula de Epley (squat_analyzer.py: $est_rm = weight_kg * (1 + 0.0333 * valid_reps)$).

Epley, B. (1985). Poundage chart. En Boyd Epley Workout (p. 86). University of Nebraska Press.

Aplicado en la validacion del criterio de profundidad IPF (angulo de rodilla ≤ 90 grados).

International Powerlifting Federation. (2023). IPF Technical Rules Book. Recuperado de https://www.powerlifting.sport/fileadmin/ipf/data/rules/technical-rules/english/IPF_Technical_Rules_Book_2023.pdf

PROMETEO

Backend Java / Spring Boot

Aplicado en la implementacion del backend BEPerfectSquat con Spring Boot 3.4.13.

VMware, Inc. (2024). Spring Boot reference documentation (v3.4). Recuperado de <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Aplicado en la capa de persistencia JPA con H2 y el mapeo de entidades AnalysisSession, Video y RepDetail.

VMware, Inc. (2024). Spring Data JPA reference documentation. Recuperado de <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>

Desarrollo Android / Jetpack Compose

Aplicado en la implementacion de la app Android con Jetpack Compose, Material3 y Navigation Compose.

Google LLC. (2024). Jetpack Compose documentation. Recuperado de <https://developer.android.com/jetpack/compose>

Aplicado en la configuracion de Retrofit 2.9.0 y OkHttp 4.12.0 para la comunicacion HTTP desde Android.

Square, Inc. (2024). Retrofit: A type-safe HTTP client for Android and Java. Recuperado de <https://square.github.io/retrofit/>

Python / FastAPI

Aplicado en la implementacion del endpoint POST /analyze del servicio PYAnalysisService.

Ramirez, S. (2024). FastAPI documentation (v0.109). Recuperado de <https://fastapi.tiangolo.com/>